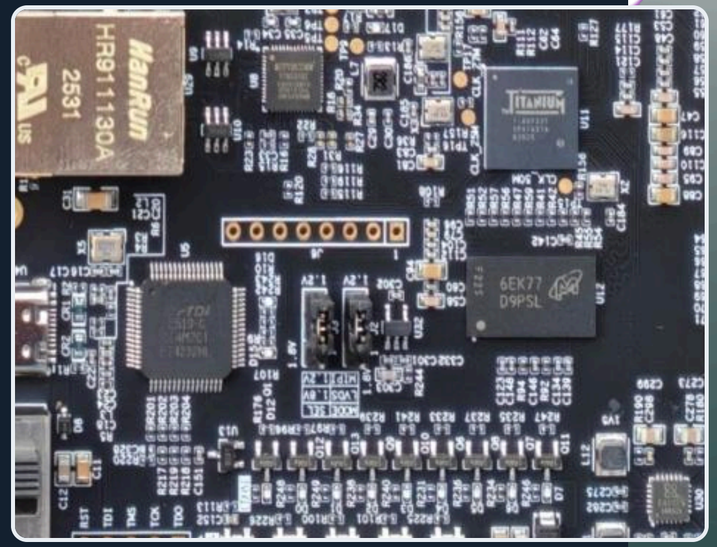
2026 嵌入式芯片与系统设计竞赛 · 易灵思 FPGA 赛道 · 赛题二

# gfx2d
# RISC-V + FPGA
# 2D 渲染加速引擎

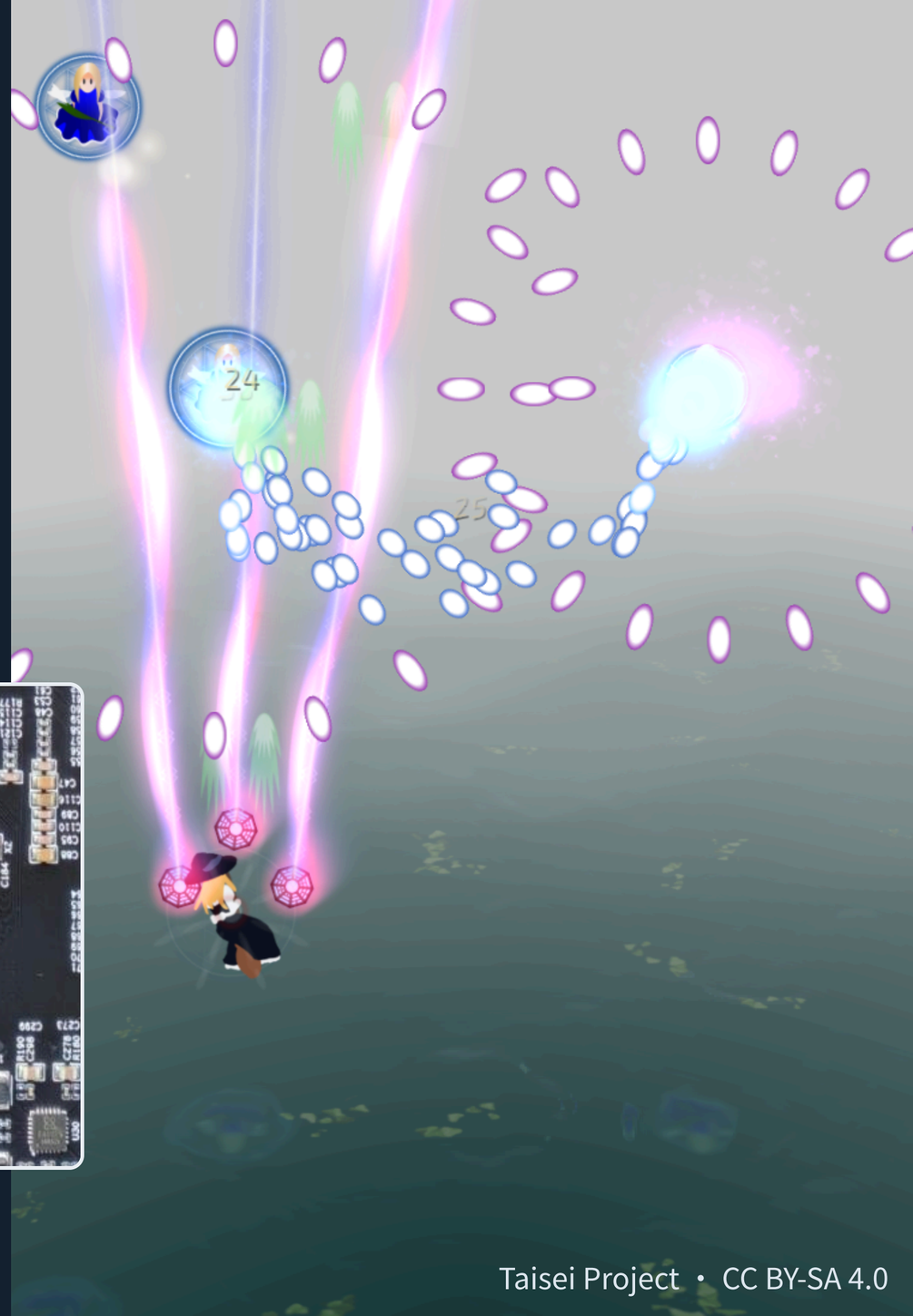



Ti60F225 · Sapphire RV32 SoC

DDR3 256 MB · HDMI 640×480@60

指标 $N_{max}$：60 FPS 稳定时的同屏 Sprite 数

[队伍名] · 2026-09-18



Taisei Project · CC BY-SA 4.0

# 赛题与指标






**游戏区：背景 · 自机 · 敌机 · 弹幕 · 粒子**
每个元素是一个 Sprite：一次纹理拷贝 + 混合

**HUD：分数 · N · 帧率**
软 / 硬件帧率同屏对比

帧率

Taisei Project（开源东方同人弹幕射击）· Wikimedia Commons · CC BY 4.0

## 赛题二：RISC-V + FPGA 2D 渲染加速引擎

### 基础要求

**命令解析**
操作 · 源 / 目的地址 · 宽高 → FILL / BLIT

**直接访存**
加速器为 AXI 主机 · 突发 · 内部 FIFO

**HDMI 输出**
≥ 640×480@60

**C 驱动 · 双缓冲**
寄存器配置 · 防撕裂

**软 / 硬件帧率实时对比**
大量方块移动 · 大图快速切换

### 高阶要求

**Alpha 混合**
DSP：fg · a + bg · (1 − a)

**Color Key**
透明色像素不写入

**高负载游戏 Demo**
弹幕射击 · 吸血鬼幸存者类

**$N_{max}$**
Alpha + Color Key 开启
60 FPS 稳定下同屏活动 Sprite 数
唯一排名指标

### 指标口径（本设计定义）

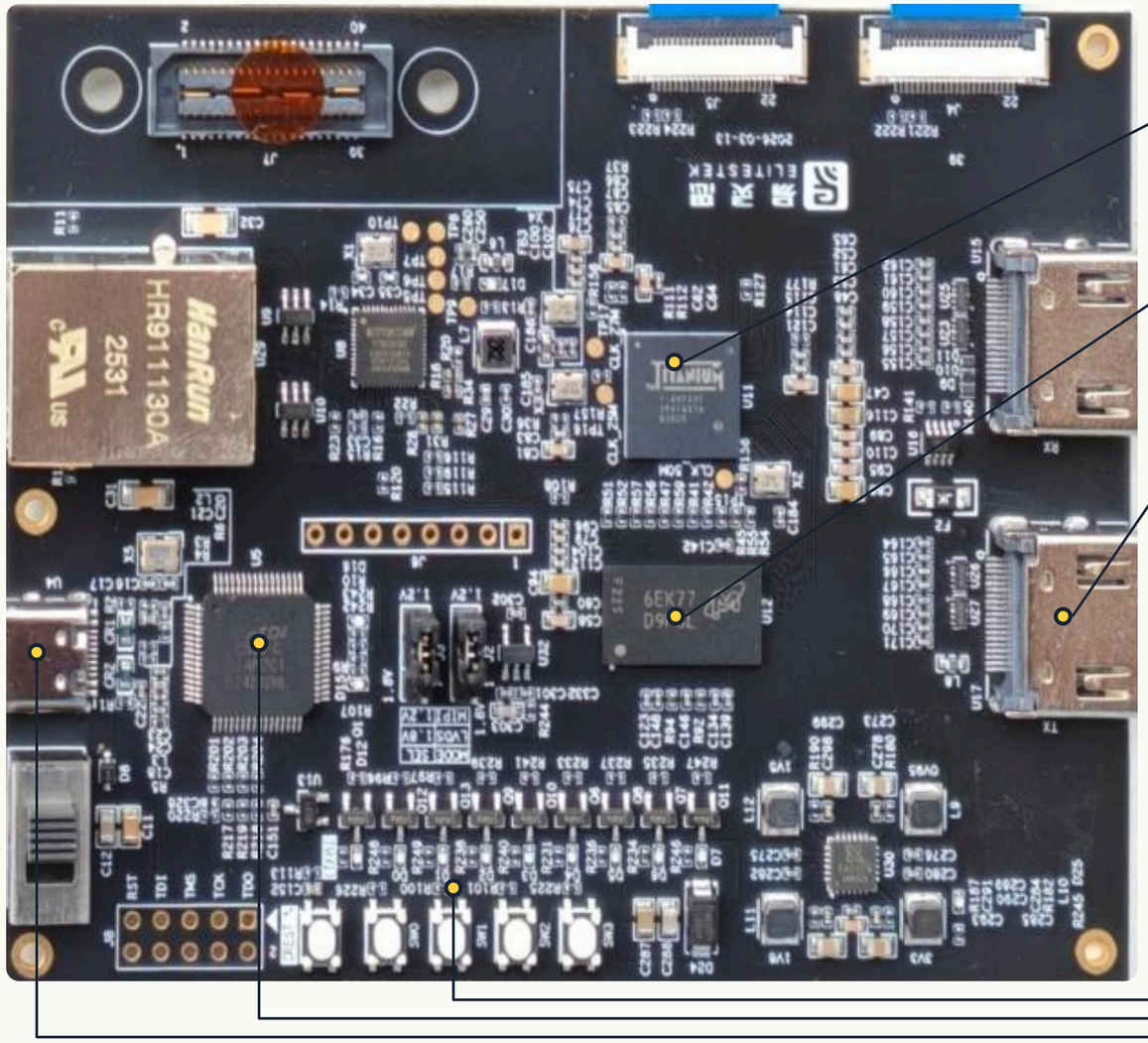
**稳定**
连续 60 s 无帧 > 16.67 ms

**活动元素**
≥ 1 像素落在屏内的绘制命令

**搜索**
逐步加量 · 掉帧回退

# Ti60F225 开发板与资源






**Ti60F225 FPGA**
60,800 XLR · 160 DSP · 4 PLL
片上 RAM 256 块 × 10 Kbit

**DDR3 256 MB**
16 bit · 软核控制器 800 Mbps

**HDMI TX**
FPGA 高速 IO 直驱 · 4 对 LVDS

**FT4232H**
B 通道 JTAG · C 通道 UART

**USB-C**
供电 · 下载 · 串口

**KEY × 4 · LED × 8**
SPI Flash W25Q64 8 MB（板上）

板卡照片：易灵思 Ti60F225 DEMO 板 V4 软硬件设计说明

## 资源占用 2026-09-17 编译

XLR 16,151 / 60,800 **26.6%**

片上 RAM 65 / 256 块 **25.4%**

DSP 4 / 160 **2.5%**

## 片上 RAM 的 65 块

| DDR3 控制器 22 | CPU 缓存 16 | DDR CDC 17 | 其余 10 |
|---|---|---|---|

## 一帧放不进片上 RAM

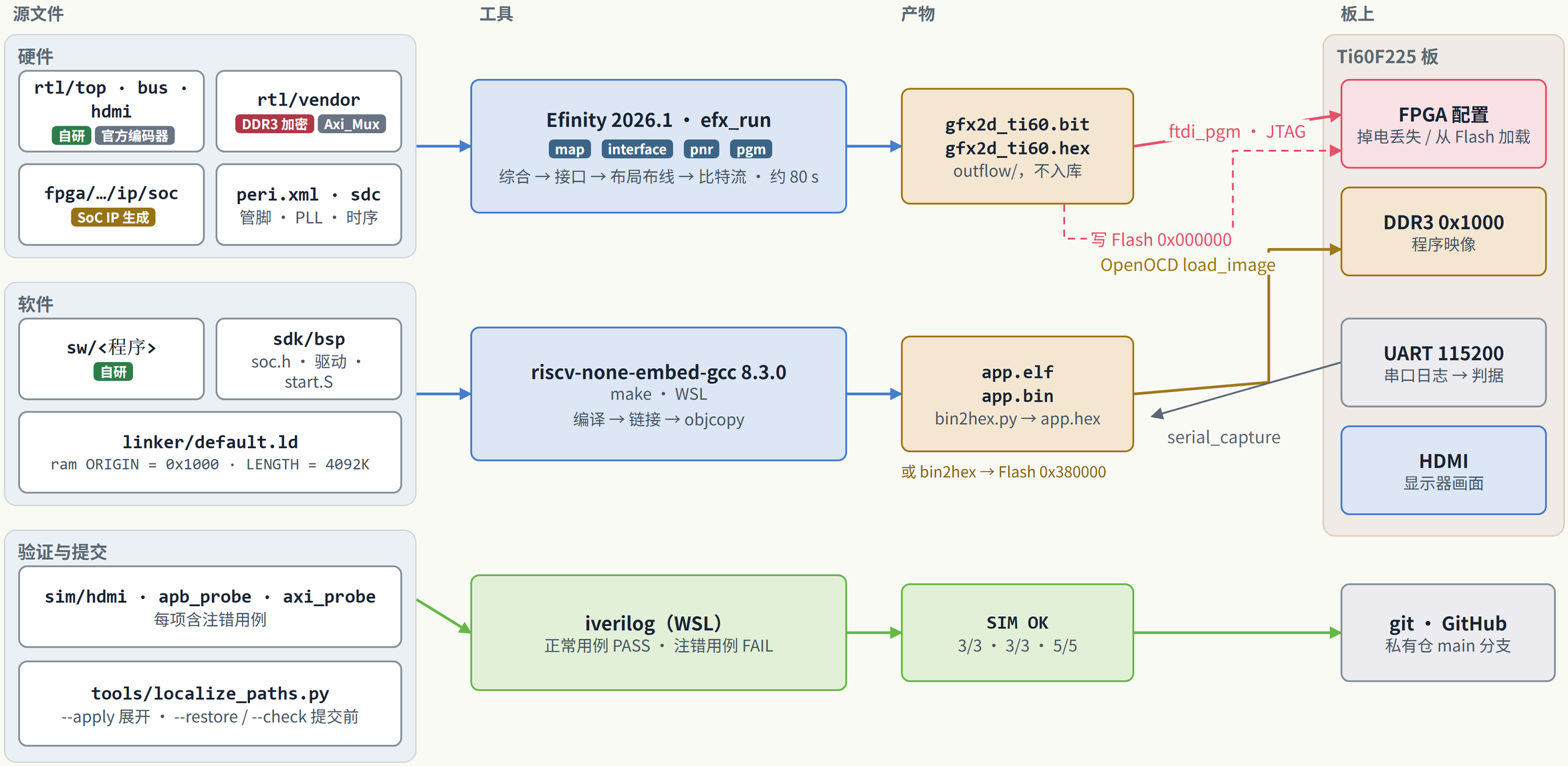
640 × 480 × 32 bit **9.83 Mbit**

全部片上 RAM 256 × 10 Kbit **2.56 Mbit**

**帧缓冲 · 纹理图集 → DDR3**
片上 RAM 只分给热纹理缓存与条带缓冲

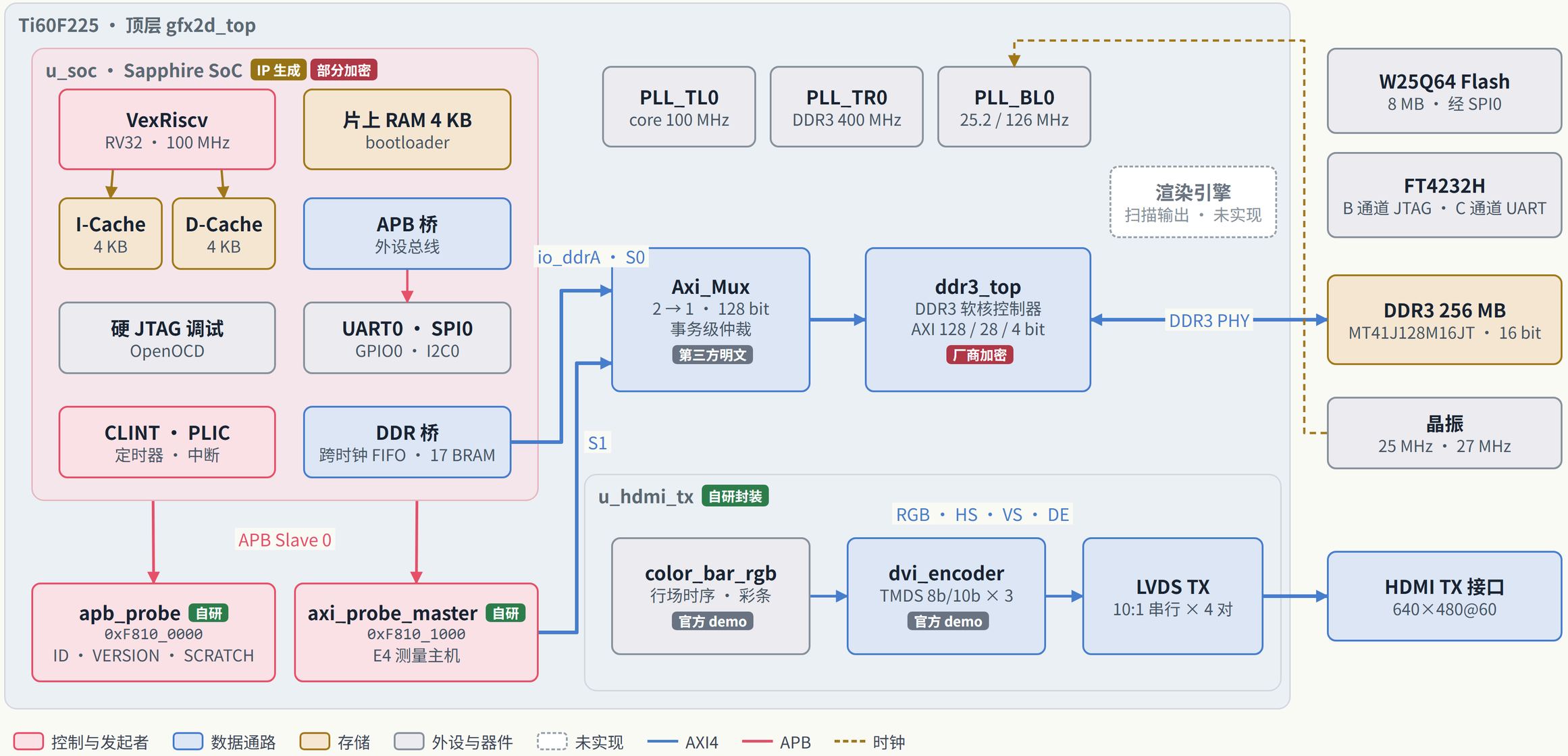
# 工程组成与构建流程




源文件
工具
产物
板上
硬件
rtl/top · bus · hdmi
自研
官方编码器
rtl/vendor
DDR3 加密
Axi_Mux
fpga/…/ip/soc
SoC IP 生成
peri.xml · sdc
管脚 · PLL · 时序
软件
sw/<程序>
自研
sdk/bsp
soc.h · 驱动 · start.S
linker/default.ld
ram ORIGIN = 0x1000 · LENGTH = 4092K
验证与提交
sim/hdmi · apb_probe · axi_probe
每项含注错用例
tools/localize_paths.py
--apply 展开 · --restore / --check 提交前
Efinity 2026.1 · efx_run
map
interface
pnr
pgm
综合 → 接口 → 布局布线 → 比特流 · 约 80 s
riscv-none-embed-gcc 8.3.0
make · WSL
编译 → 链接 → objcopy
iverilog（WSL）
正常用例 PASS · 注错用例 FAIL
gfx2d_ti60.bit
gfx2d_ti60.hex
outflow/，不入库
app.elf
app.bin
bin2hex.py → app.hex
或 bin2hex → Flash 0x380000
SIM OK
3/3 · 3/3 · 5/5
ftdi_pgm · JTAG
写 Flash 0x000000
OpenOCD load_image
serial_capture
Ti60F225 板
FPGA 配置
掉电丢失 / 从 Flash 加载
DDR3 0x1000
程序映像
UART 115200
串口日志 → 判据
HDMI
显示器画面
git · GitHub
私有仓 main 分支

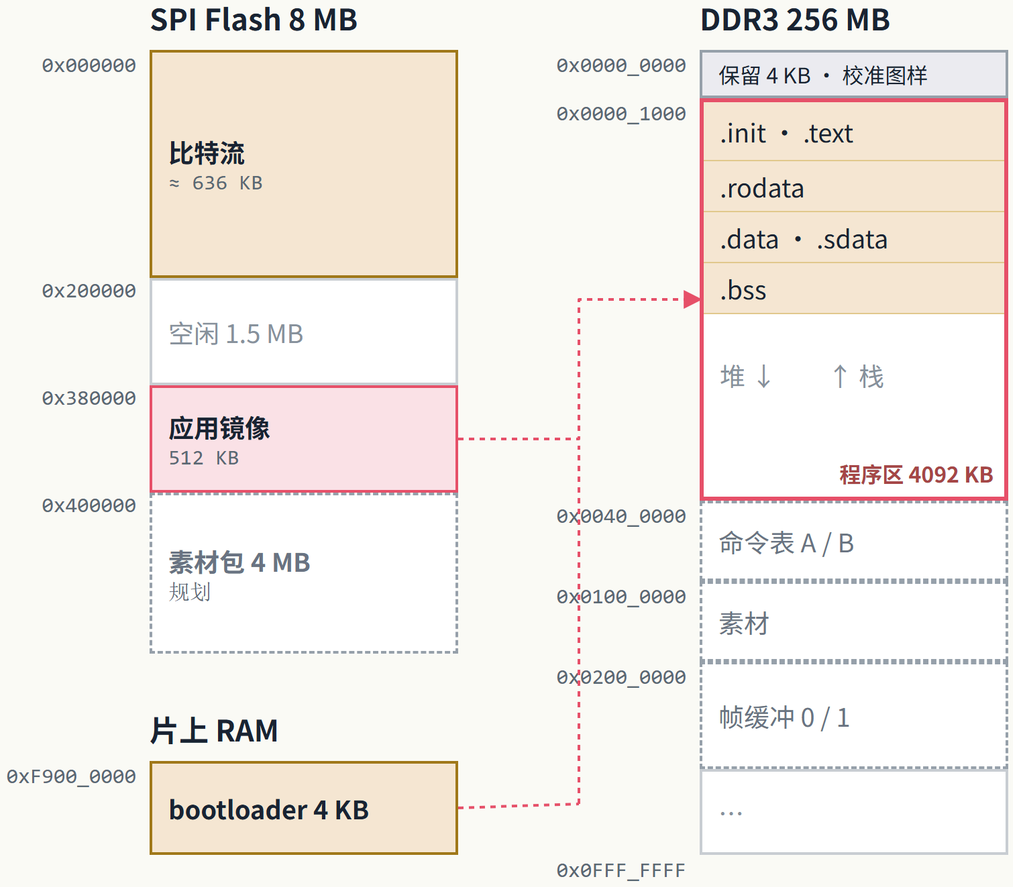
# 当前硬件架构（已上板）

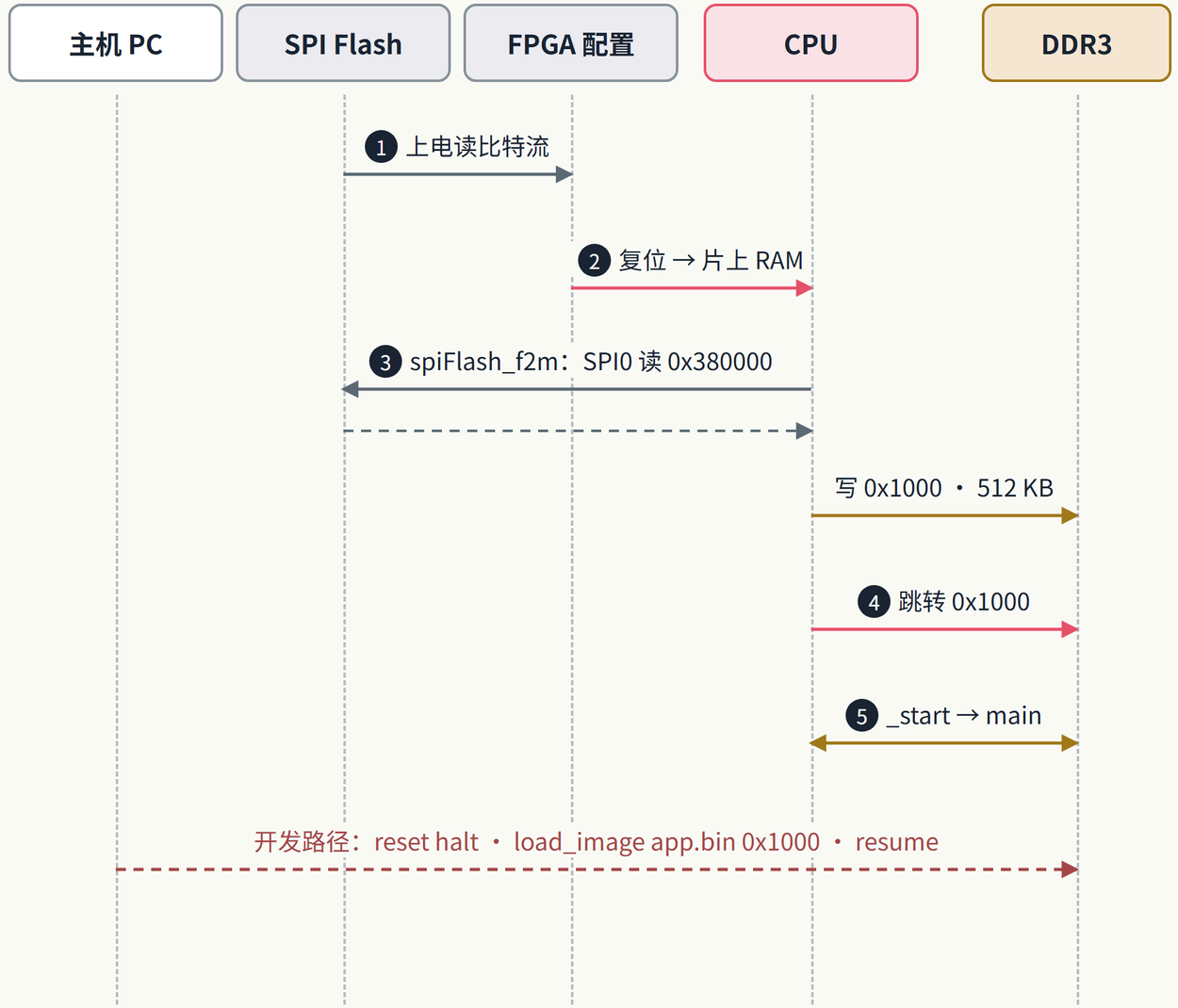



Ti60F225 · 顶层 gfx2d_top
u_soc · Sapphire SoC
IP 生成
部分加密
VexRiscv
RV32 · 100 MHz
片上 RAM 4 KB
bootloader
I-Cache
4 KB
D-Cache
4 KB
APB 桥
外设总线
硬 JTAG 调试
OpenOCD
UART0 · SPI0
GPIO0 · I2C0
CLINT · PLIC
定时器 · 中断
DDR 桥
跨时钟 FIFO · 17 BRAM
APB Slave 0
apb_probe
自研
0xF810_0000
ID · VERSION · SCRATCH
axi_probe_master
自研
0xF810_1000
E4 测量主机
PLL_TL0
core 100 MHz
PLL_TR0
DDR3 400 MHz
PLL_BL0
25.2 / 126 MHz
渲染引擎
扫描输出 · 未实现
io_ddrA · S0
S1
Axi_Mux
2→1 · 128 bit
事务级仲裁
第三方明文
ddr3_top
DDR3 软核控制器
AXI 128 / 28 / 4 bit
厂商加密
DDR3 PHY
u_hdmi_tx
自研封装
RGB · HS · VS · DE
color_bar_rgb
行场时序 · 彩条
官方 demo
dvi_encoder
TMDS 8b/10b × 3
官方 demo
LVDS TX
10:1 串行 × 4 对
W25Q64 Flash
8 MB · 经 SPI0
FT4232H
B 通道 JTAG · C 通道 UART
DDR3 256 MB
MT41J128M16JT · 16 bit
晶振
25 MHz · 27 MHz
HDMI TX 接口
640×480@60
控制与发起者
数据通路
存储
外设与器件
未实现
AXI4
APB
时钟

# 启动：从 Flash 到 main

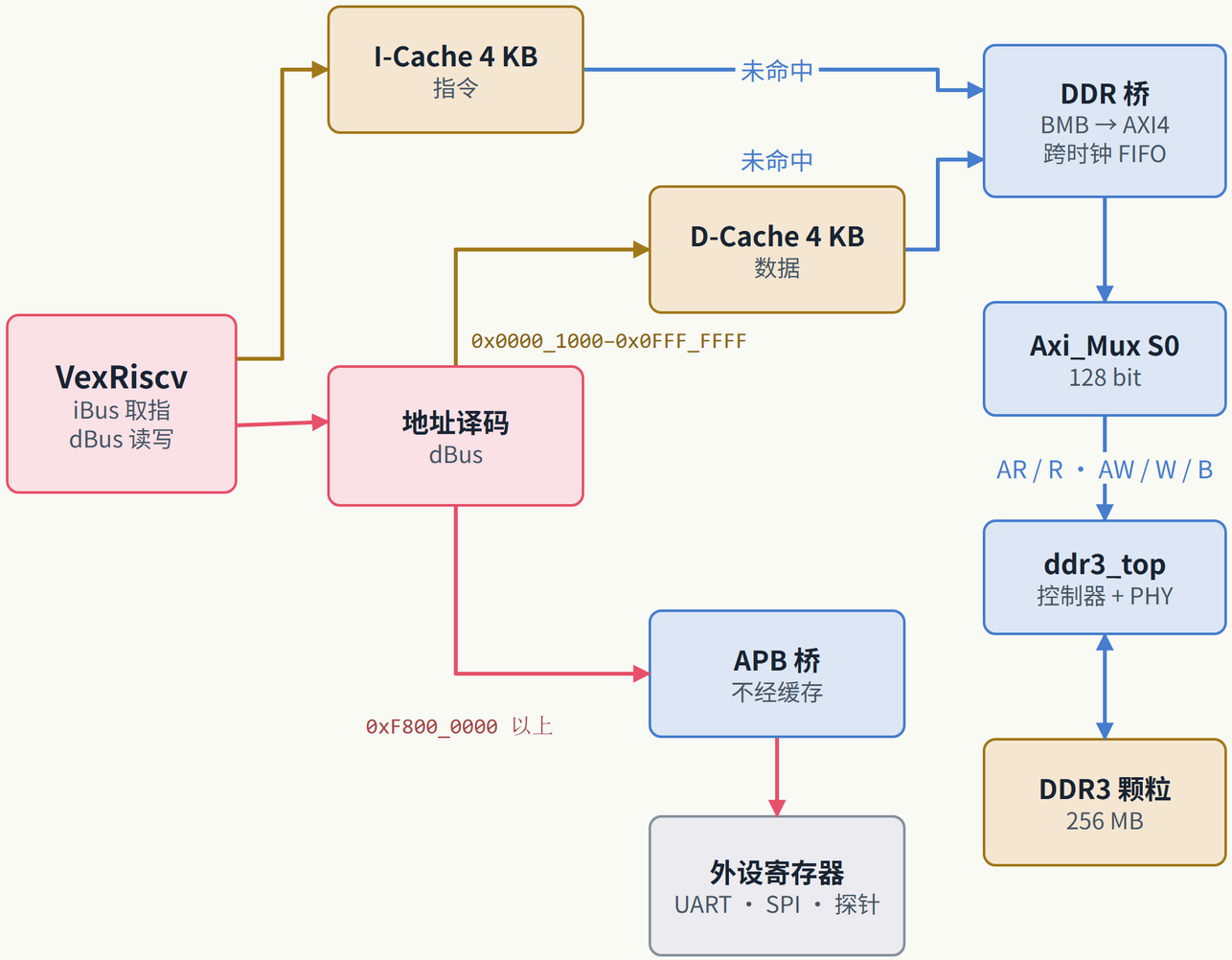




# CPU 取指与访存






## D-Cache：1 路直接映射 · 64 行 × 64 B




CPU 访问的 32 位地址

| 行 | V | 标签 | 数据 64 B |
|---|---|---|---|
| 0 | 1 | 0x00012 | … |
| 1 | 0 | — | — |
| k | 1 | tag ?= 地址[31:12] | 命中：直接返回 |
| ⋮ | ⋮ | ⋮ | ⋮ |
| 63 | 1 | 0x0C800 | … |

**读 · 未命中**
整行 64 B 从 DDR3 装入
装入一行约 69 周期

**写**
直接写 DDR3，不装行
SoC 没有写缓冲

**第二个主机改写 DDR3**
缓存副本不会自动失效
CPU 读回旧值

**读回前先失效**
单地址 4 周期 · 全清 67 周期
失效须晚于外部写完成

实验 E2 / E3 板上实测（均已验收）· platform 第 02、05 章

# MMIO、驱动与中断



## CPU 地址空间（不按比例）

| | |
|---|---|
| 0x0000_1000 | **DDR3 256 MB** |
| | 低 28 位译码，以上回绕 |
| 0xE100_0000 | AXI 从口窗口 · 未用 |
| 0xF801_0000 | UART0 |
| 0xF801_4000 | SPI0（Flash） |
| 0xF801_5000 | GPIO0（LED） |
| 0xF810_0000 | **apb_probe** |
| 0xF810_1000 | **axi_probe_master** |
| 0xF8B0_0000 | CLINT 定时器 |
| 0xF8C0_0000 | PLIC 中断控制器 |
| 0xF900_0000 | 片上 RAM |

来源：sdk/bsp/⋯/include/soc.h

## 驱动访问寄存器

```
#define PROBE  0xF8100000
#define REG(o) (*(volatile uint32_t *)(PROBE + (o)))

uint32_t id = REG(0x000);  // 板上读到 0x47465832 "GFX2"
REG(0x008) = 0xA5A55A5A;   // SCRATCH 读回一致
REG(0x100) = 0;            // 未定义地址：PSLVERR
```

**应用**
调用驱动函数

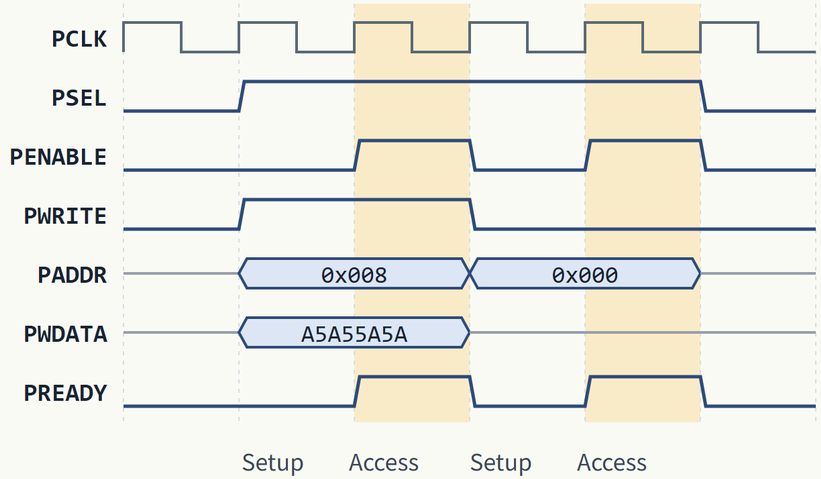
→

**驱动**
寄存器表 · ISR

→

**dBus → APB 桥**
译码 → PSEL

## APB 传输：Setup → Access




## apb_probe 寄存器

| 偏移 | 名称 | 读写 | 内容 |
|---|---|---|---|
| 0x000 | ID | RO | 0x47465832 |
| 0x004 | VERSION | RO | 0x00000001 |
| 0x008 | SCRATCH | RW | 任意 32 位 |
| 其余 | — | — | 读 0 · PSLVERR |

## 中断路径

**外设中断线**
UART0=1 · SPI0=4 · 用户 A=16

↓

**PLIC**
门控 · 优先级 · 使能

↓

**CPU：MEIP → mtvec**
陷入异常入口

↓

**claim → ISR → complete**
驱动的中断服务函数

### 引擎规划的中断

用户中断 A（16）

- 场同步
- 命令表执行完
- 错误（粘滞位 + 序号）

轮询：CPU 反复读 STATUS
中断：外设主动通知

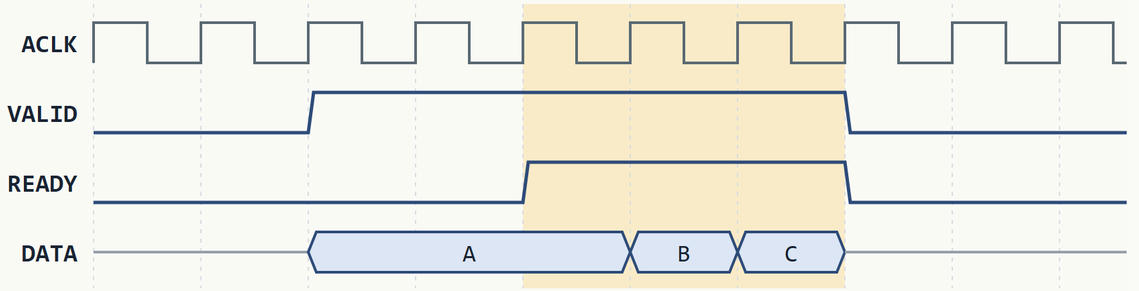
# AXI4 总线



## 五个独立通道

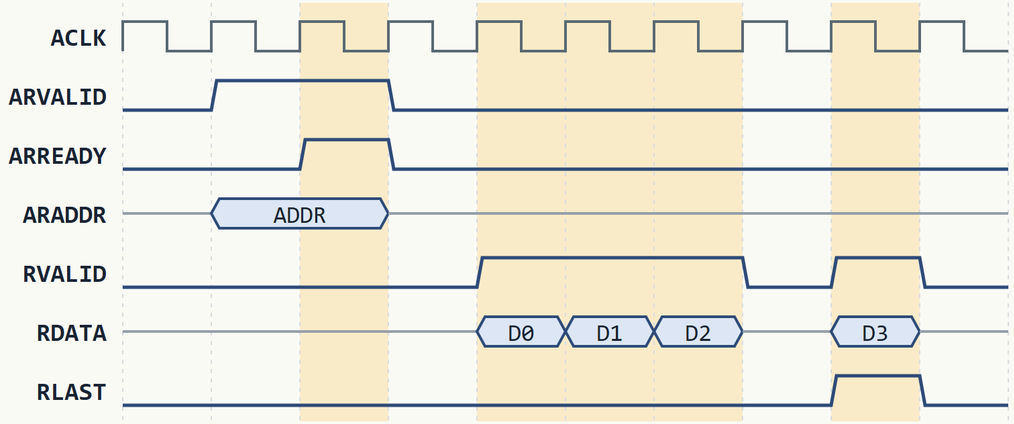



## 握手：同一上升沿 VALID 与 READY 同为 1，传输一拍




VALID 拉高后保持到握手，不能等 READY
READY 可以随时反压；黄色列 = 传输成立（A、B、C）
一拍 128 bit = 16 B = 4 个 32 位像素

## INCR 突发读：一次地址，多拍数据




**本例**
ARLEN = 3 → 4 拍
D0 @ ADDR
D1 @ ADDR+16
D2 @ ADDR+32
D3 @ ADDR+48

第 7 拍从机未就绪，数据停顿

## 本工程的 AXI4 拓扑




### 4 KB 边界：突发不能跨页



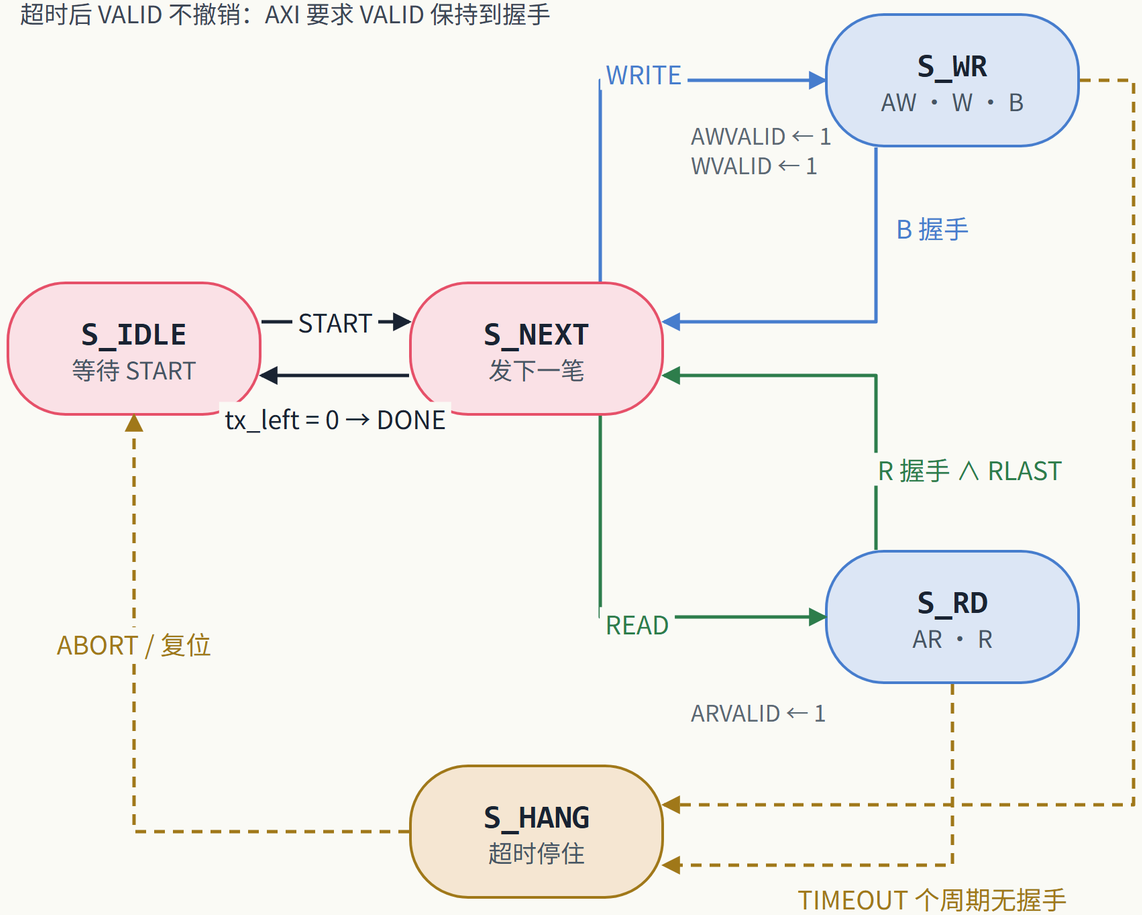

实测：突发上限 256 拍；帧缓冲每行占满一页

# 有限状态机：以 axi_probe_master 为例

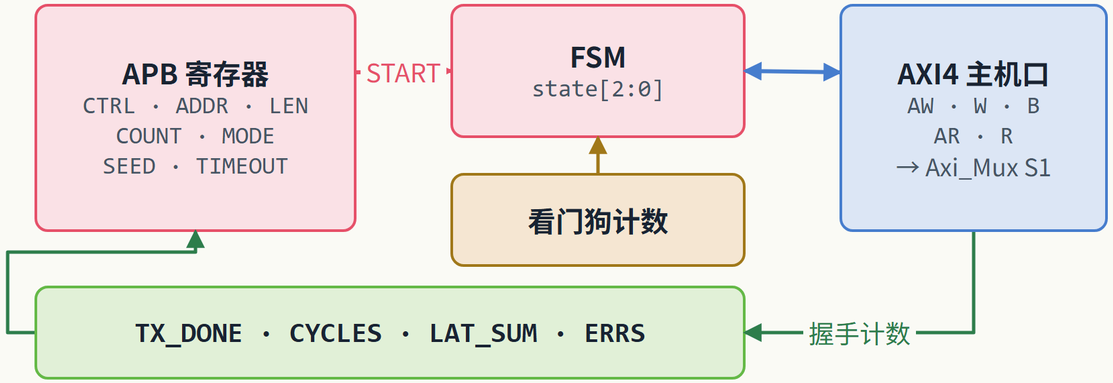


## axi_probe_master 状态机 rtl/bus/axi_probe_master.v

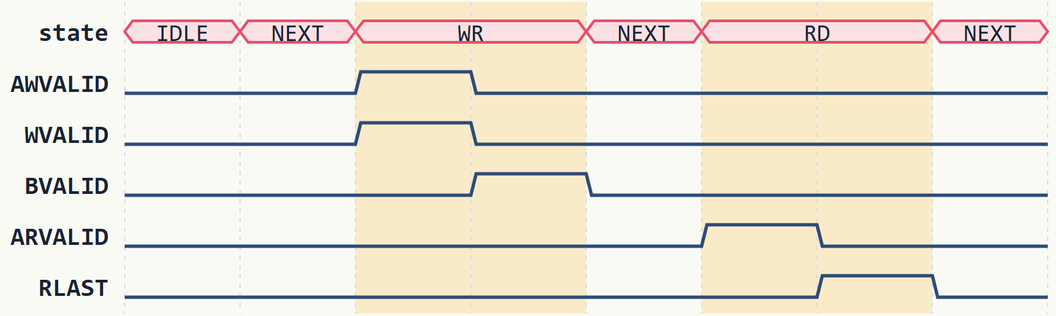
状态寄存器 3 bit，每个上升沿至多转移一次
VALID 在进入状态前置位，握手后撤销
超时后 VALID 不撤销：AXI 要求 VALID 保持到握手




## 模块结构




```
S_NEXT: if (tx_left == 0) begin st_done <= 1; state <= S_IDLE; end
        else if (cfg_write) begin
          awaddr <= cur_addr; awvalid <= 1; wvalid <= 1;
          state <= S_WR; end
        else begin araddr <= cur_addr; arvalid <= 1; state <= S_RD; end
S_RD:   if (ar_fire) arvalid <= 0;
        if (r_fire && rlast) begin
          tx_left <= tx_left - 1; state <= S_NEXT; end
```



# HDMI 显示链路



## 一帧的扫描：800 × 525（按比例）

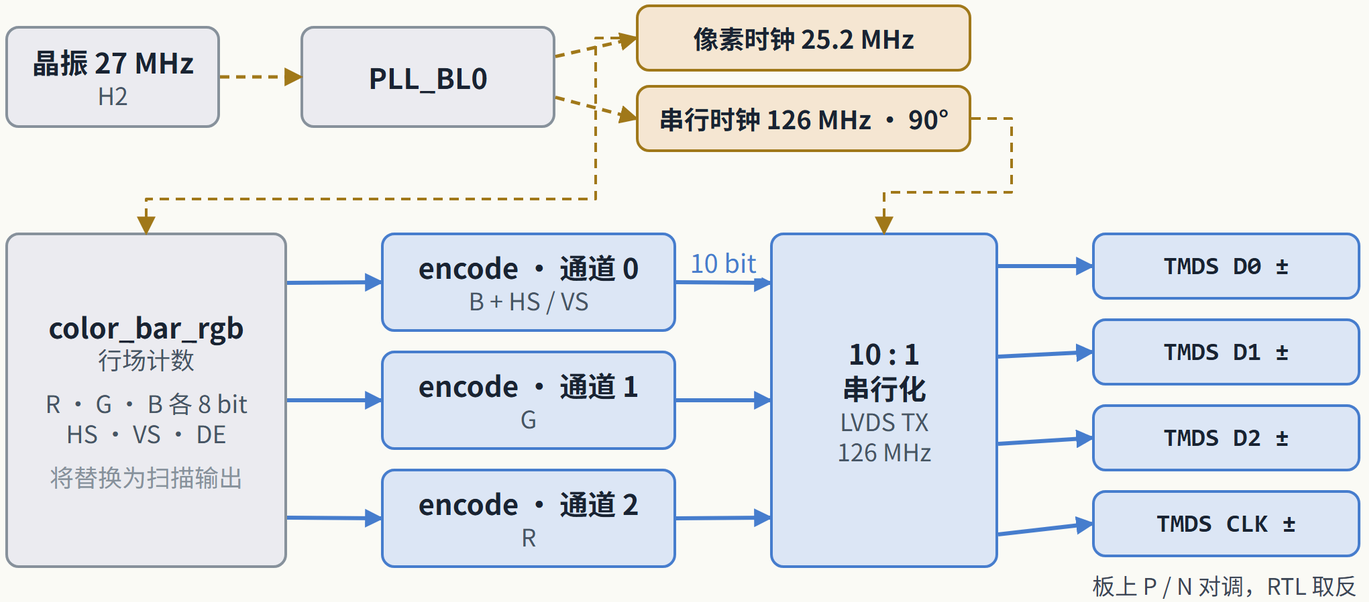



| | 可见 | 前沿 | 同步 | 后沿 | 总计 |
|---|---|---|---|---|---|
| 水平 / 像素 | 640 | 16 | 96 | 48 | **800** |
| 垂直 / 行 | 480 | 10 | 2 | 33 | **525** |

**800 × 525 × 60 Hz = 25.2 MHz**
像素时钟；标准值 25.175 MHz，差 0.1%，显示器容差内

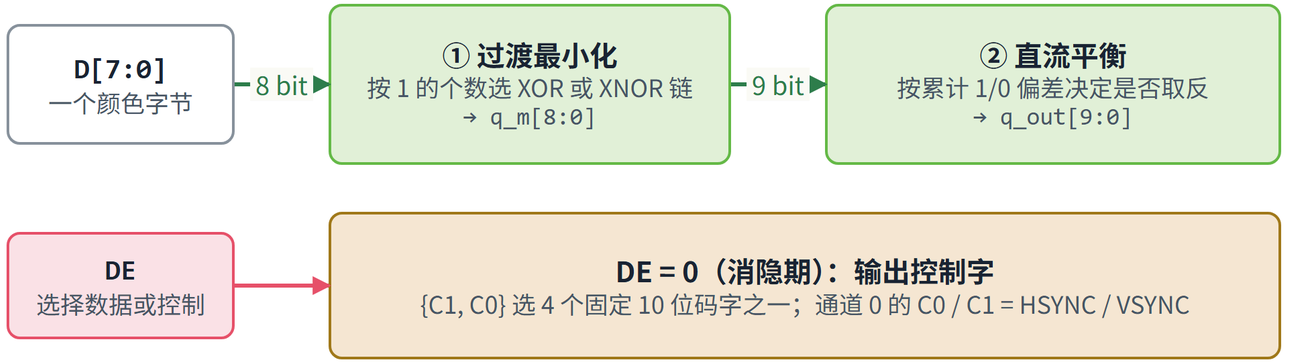
参数取自 rtl/hdmi/hdmi_tx.v · 同步脉冲低有效

## 发送链路




板上 P / N 对调，RTL 取反

### TMDS 8b/10b 编码 · encode.v



# 帧缓冲、撕裂与双缓冲

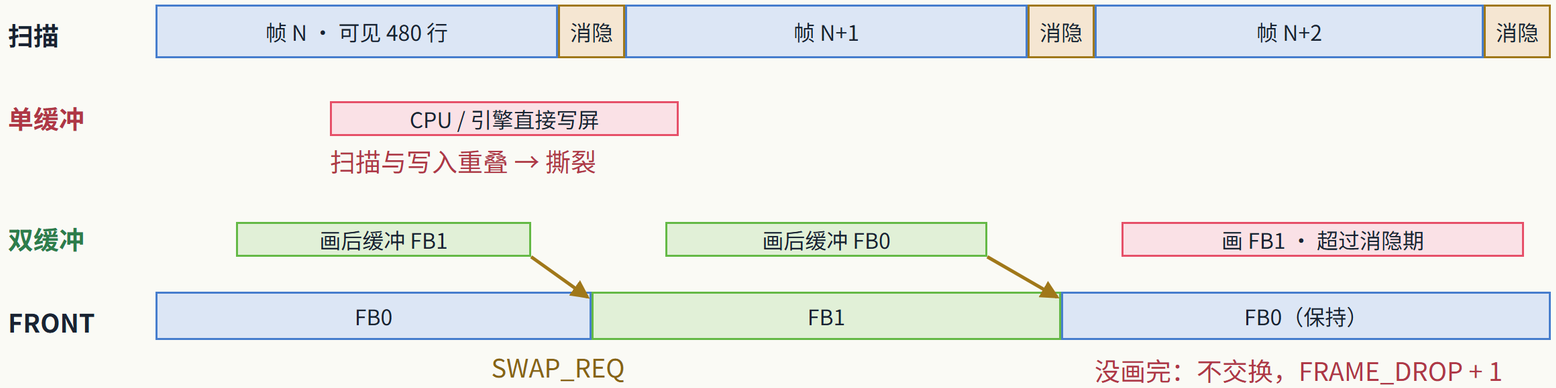


## 画面撕裂



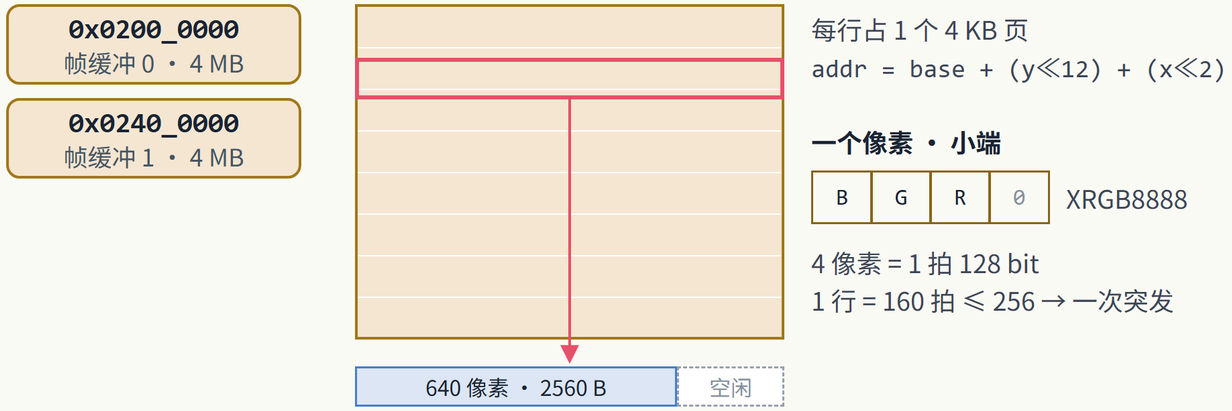

扫描到一半时帧缓冲被改写，一屏里出现两帧

## 单缓冲与双缓冲

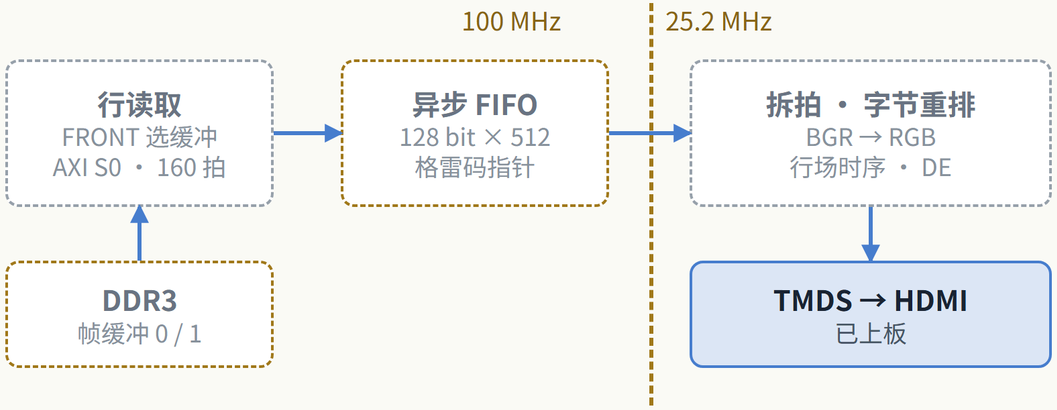



交换只在场消隐期生效；消隐 45 行 ≈ 1.43 ms
CPU 填第 N 帧命令表时，引擎在画第 N−1 帧

## 帧缓冲布局（DDR3）



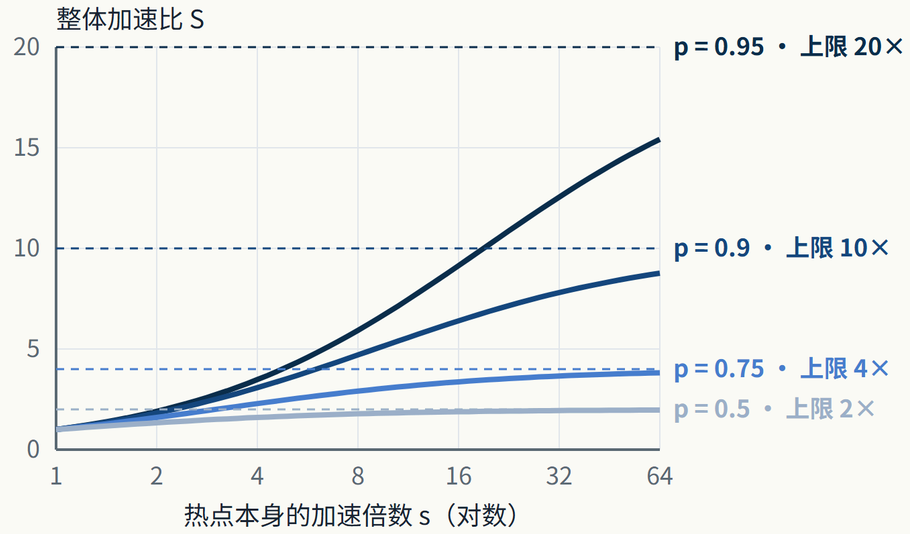

## 扫描输出（规划）



# 加速器设计方法



## Amdahl 定律：S = 1 / ((1 − p) + p / s)

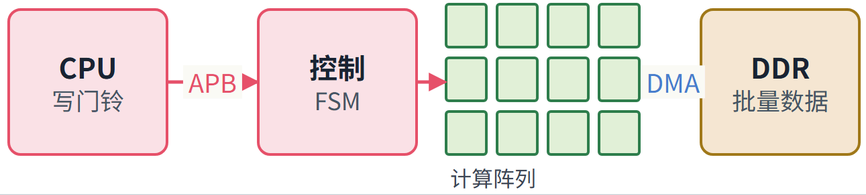



## 两种硬件形态

### CPU 内协处理器



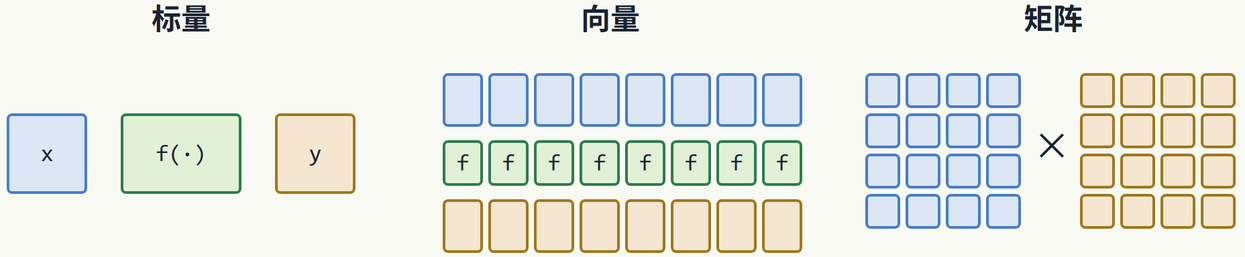

### SoC 外设加速器




## 流程

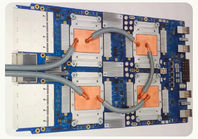



## 三类计算




| | 标量 | 向量 | 矩阵 |
|---|---|---|---|
| **一次运算** | 一个数 | 一维序列，逐元素同一操作 | 二维表，行列交叉 |
| **典型运算** | 浮点乘除 · sin · exp · rsqrt | 像素混合 · 激活 · 前缀和与递推 | 全连接层 · 矩阵乘 |
| **合适形态** | CPU 内协处理器 | 外设 + DMA | 外设 + DMA + 计算阵列 |
| **以往实践** | 标量浮点协处理器 | 向量流 · 线性递推加速器 | 矩阵乘加速器 ≈ TPU v1 |

本题的像素混合属于向量类：逐像素运算简单、数量大，形态选外设加速器，设计重心在访存



Google Cloud TPU v3 · Zinskauf · Wikimedia Commons · CC BY-SA 4.0

# 本题拆账



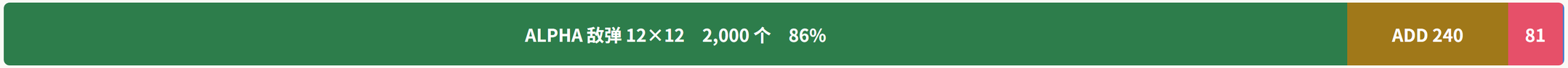
## 场景 S1：Boss 弹幕段一帧（400×480 游戏区，估算）

**按像素**



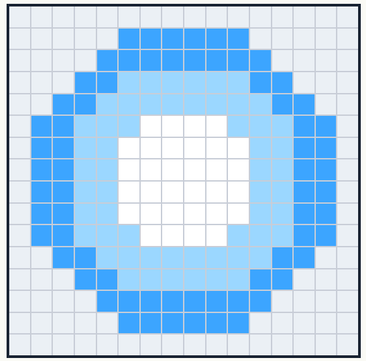

合计约 68.3 万像素 ≈ 2.2 屏 · COPY + 色键 29,184 · FILL ≈ 2,000

**按个数**




合计约 2,320 个 Sprite · 敌机 · 自机 · HUD 文字为色键 COPY

## 一个 Sprite 在直通模式下的代价



16 × 16 纹理 · 256 像素

**每个像素**

读 src | 读 dst | × 3 | + 3 | ÷255 | 写

算术每周期 4 像素：一帧全部乘法约 7.2 万周期

**每一行**

AXI 读纹理行 | AXI 读目标行 | AXI 写目标行

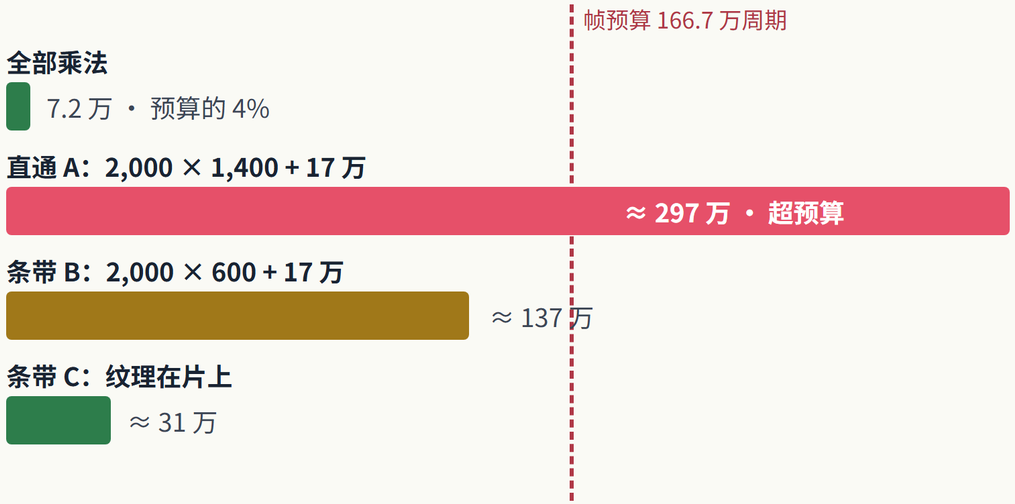
16 行 × 3 = **48 次 DDR 事务** / Sprite

读事务固定开销 27.4 周期（实测），短突发时固定开销占主导

**算力足够，瓶颈在访存事务次数**

加速方向：减少每个 Sprite 的 DDR3 事务 → 片上纹理缓存 + 条带合成

## 一帧 16.67 ms 的周期预算（100 MHz）

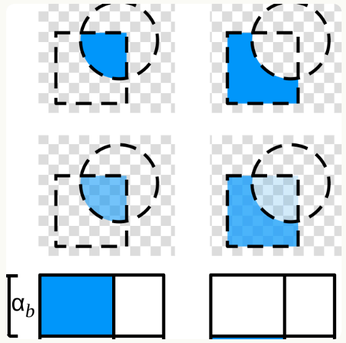



A、B、C 为设计文档 3.1 节按 t_tx = 20 周期的估算；E4 实测读 27.4、写 7.3，排序不变

# Alpha 混合、线性递推与色键



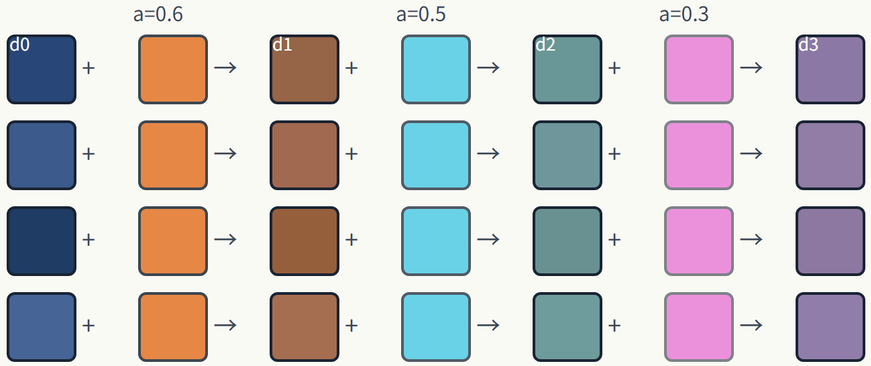
## Alpha 合成




Porter–Duff「A over B」上：不透明；下：半透明 · Wikimedia Commons · 公有领域

## 同一像素上的多层：一阶线性递推

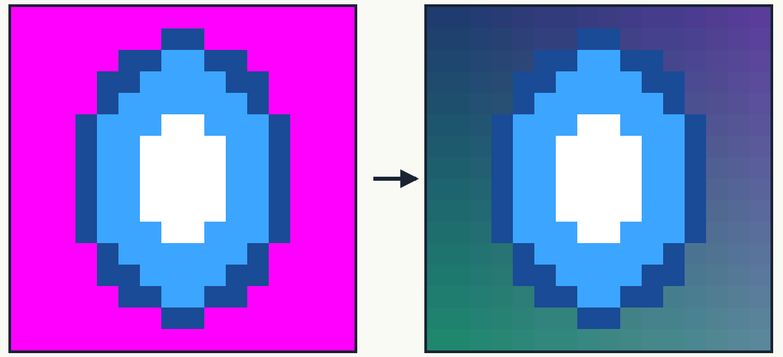
$$d_k = s'_k + (1 - a_k) \cdot d_{k-1} \qquad s' = s \cdot a \text{（预乘）}$$




↕ 像素之间无依赖：一拍 4 像素并行（向量运算）

→ 层之间有依赖：按命令表顺序串行（递推）

## Color Key



纹理：品红 #FF00FF 为透明色　　写入后：透明色像素保持目的值

```
if (key_en && src_word == key_word) skip_write;
```

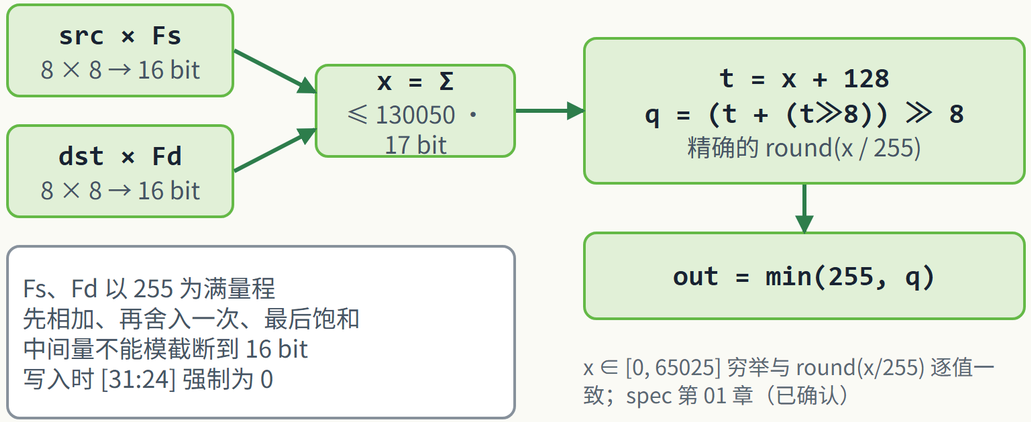
比较源表面的 32 位存储字；与混合方程互不影响

## 混合方程表：out = src · Fs + dst · Fd

| 方程 | Fs | Fd | src | dst | out | 用途 |
|---|---|---|---|---|---|---|
| COPY | 1 | 0 | | | | 背景、不透明图块 |
| ALPHA | 1 | 1 − a | | | | 半透明弹幕 |
| ADD | 1 | 1 | | | | 发光、爆炸 |
| CONST | c | 1 − c | | | | 淡入淡出 |
| MUL | dst | 0 | | | | 阴影（预留） |

示例 src = (255, 140, 40)、a = 0.6 预乘；dst = (40, 90, 200)；CONST 取 c = 0.5

## 单通道算术（R、G、B 各一份）

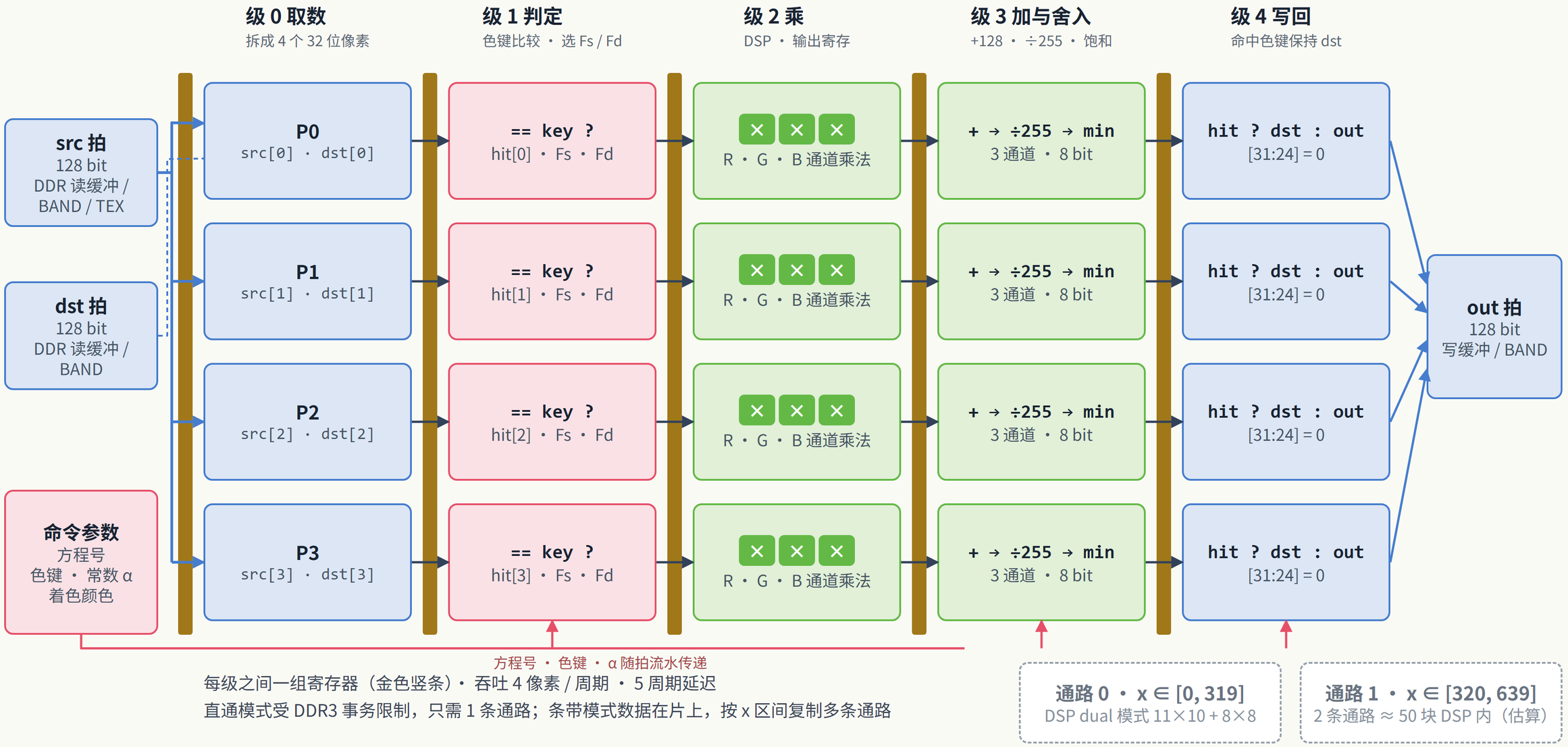



Fs、Fd 以 255 为满量程
先相加、再舍入一次、最后饱和
中间量不能模截断到 16 bit
写入时 [31:24] 强制为 0

x ∈ [0, 65025] 穷举与 round(x/255) 逐值一致；spec 第 01 章（已确认）

# 像素流水线






每级之间一组寄存器（金色竖条）· 吞吐 4 像素 / 周期 · 5 周期延迟

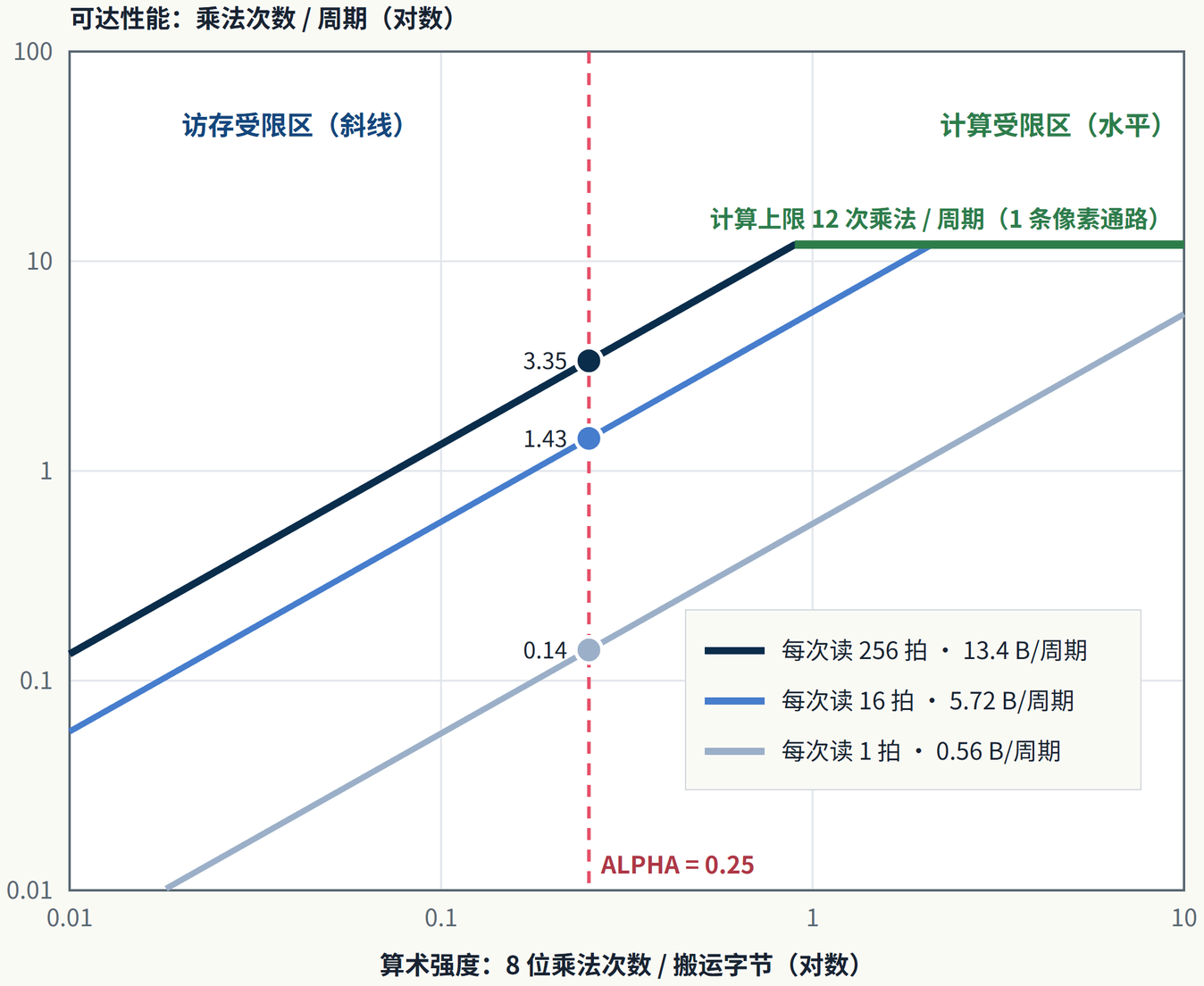
直通模式受 DDR3 事务限制，只需 1 条通路；条带模式数据在片上，按 x 区间复制多条通路

**通路 0 · x ∈ [0, 319]**
DSP dual 模式 11×10 + 8×8

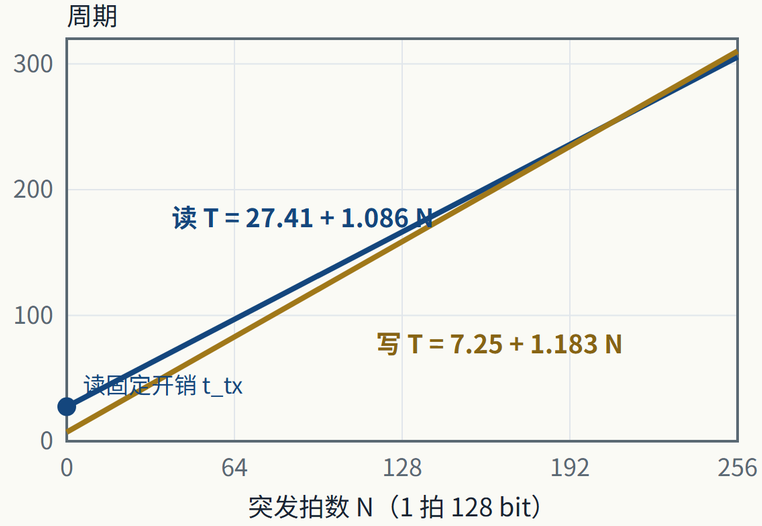
**通路 1 · x ∈ [320, 639]**
2 条通路 ≈ 50 块 DSP 内（估算）

# Roofline：访存受限






**E4 实测：每笔事务的周期数**



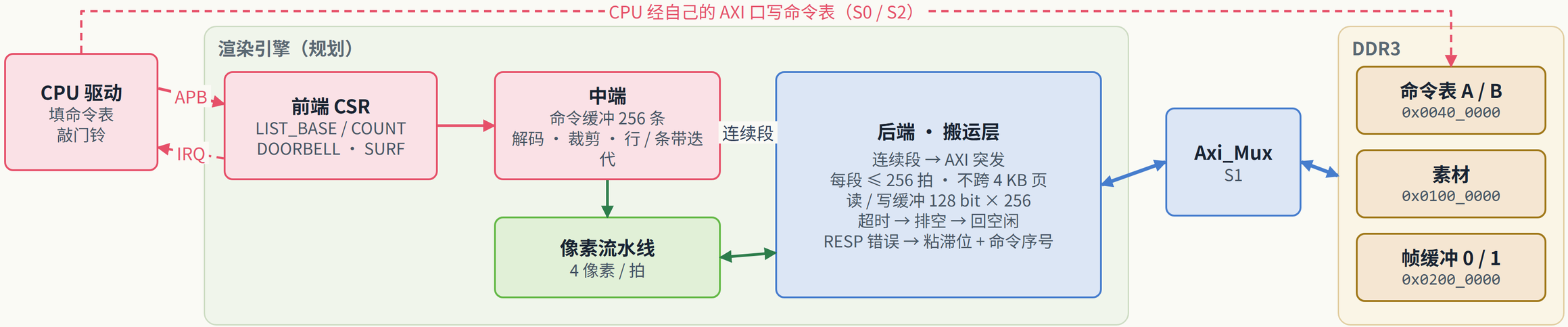

| 读 N 拍 | 拍 / 周期 | 字节 / 周期 | ALPHA 乘法 / 周期 |
|---|---|---|---|
| 1 | 0.035 | 0.56 | 0.14 |
| 16 | 0.357 | 5.7 | 1.43 |
| 256 | 0.838 | 13.4 | 3.35 |
| 计算上限 | — | — | 12（1 条通路） |

算术强度 = 每像素 3 次乘法 / 搬 12 字节 = 0.25
256 拍时工作点仍在斜线段，加乘法器无收益
单拍读的有效带宽只有 256 拍突发的 1/24

Roofline 模型：Williams, Waterman, Patterson, CACM 2009。按突发长度分出多条带宽线为本设计的扩展；带宽由 E4 拟合式推算

# DMA 与命令描述符






## 命令记录：固定 32 字节 = 8 个字

| | 31 | | | | | | | 0 |
|---|---|---|---|---|---|---|---|---|
| w0 | 常数 α | 目的表面 | 源表面 | — | 着 | 键 | 方程 | 操作码 |
| w1 | 源 y | | | | 源 x | | | |
| w2 | 目的 y（有符号） | | | | 目的 x（有符号） | | | |
| w3 | 高 | | | | 宽 | | | |
| w4 | FILL 颜色 / 着色颜色 | | | | | | | |
| w5 | 色键颜色（按源表面格式） | | | | | | | |
| w6 | 预留：翻转 · 帧号 | | | | | | | |
| w7 | 序号：调试与错误定位 | | | | | | | |

操作码：0 NOP · 1 FILL · 2 BLIT · 3 SET_CLIP · 4 BAND · 0xFF END

## 表面表：一切地址间接寻址

| base · 32 | stride_log2 · 5 | width / height | space | format |
|---|---|---|---|---|

space：DDR · TEX 纹理缓存 · BAND 条带缓冲　format：XRGB · 预乘 ARGB

```
addr = base + (y ≪ stride_log2) + (x ≪ 2)
cmd[k] = LIST_BASE + (k ≪ 5)
```

记录长度取 2 的幂 → 地址只用移位；硬件统一裁剪与越界检查

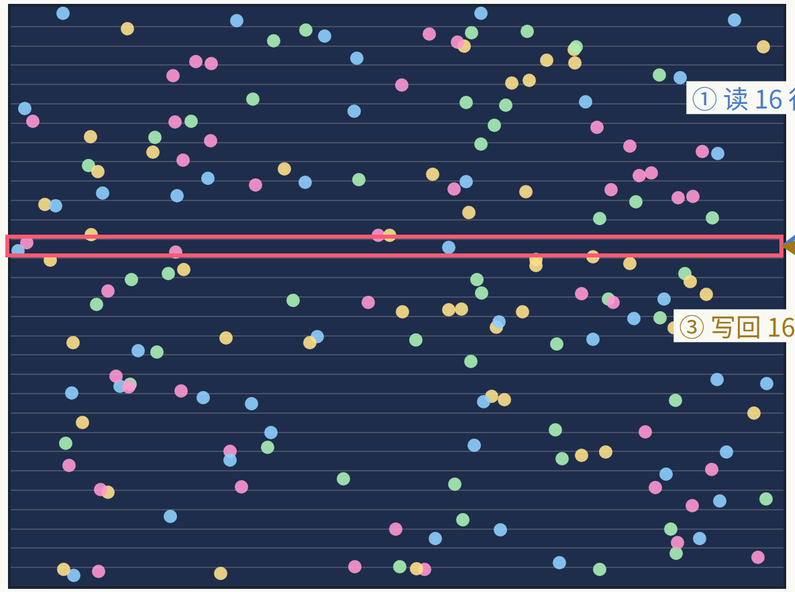
**纹理上传**
目的 space = TEX 的一条 BLIT
不另设上传接口

**计数器可对账**
RD_BEAT = Σ 读突发长度
CMD · PIXEL · WAIT_DDR
FRAME_DROP · 场同步时快照

# 片上缓存与条带合成



## 条带合成：屏幕按 16 行切成 30 条



640 × 480 后缓冲 · 圆点为 Sprite · 红框为第 k 条带

① 读 16 行 · 16 次突发

③ 写回 16 行 · 16 次突发

**BAND 条带缓冲**
640 × 16 × 32 bit
32 块 BRAM

**TEX 热纹理缓存**
128 × 128 · 52 块

② 执行本条带全部命令

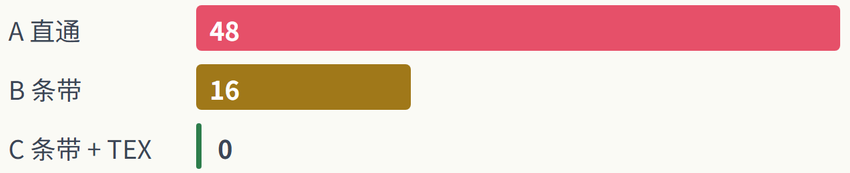
**像素流水线**
× 2 通路

**CPU：按条带给命令分桶**
高 ≤ 16 的 Sprite 最多落进 2 个条带，分桶 O(N)
同一条带内保持提交顺序 = 绘制顺序

**每帧 30 条带 × (16 读 + 16 写) = 960 次突发**
与 Sprite 数量无关；Sprite 数只受流水线算力限制

## 三种架构（估算）

### 每个 Sprite 的 DDR3 事务

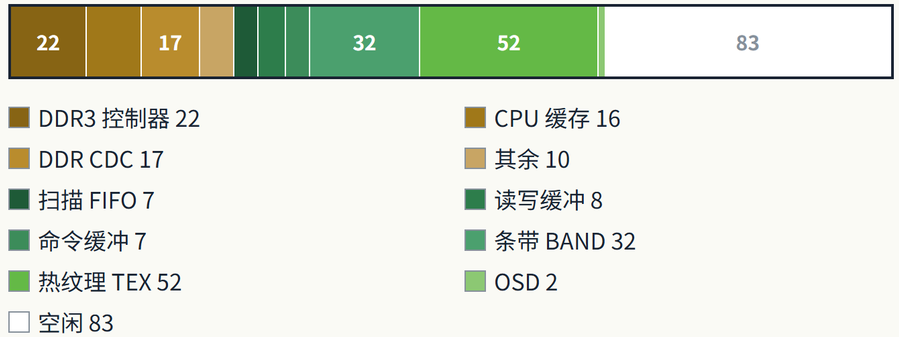



### $N_{max}$ 硬件侧上限（对数刻度）




按 t_tx = 20 周期估算（设计文档 3.1 节）；E4 实测读 27.4、写 7.3，排序不变。C 的上限超过 CPU 生成命令的速度

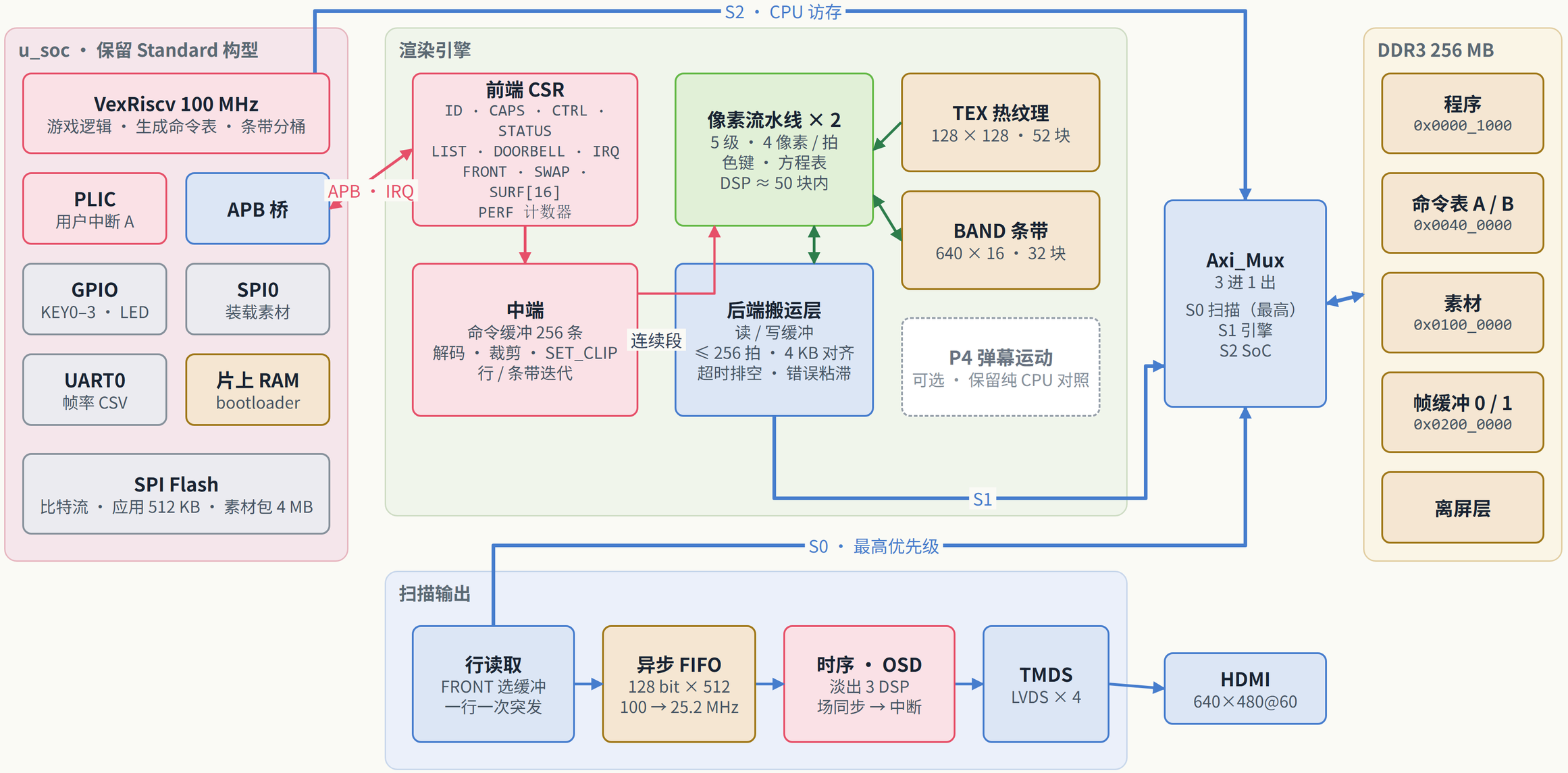
## 片上 RAM 预算：256 块




**现有 65 + 新增 108 = 173 块（67.6%），余 83 块**

# 目标硬件架构

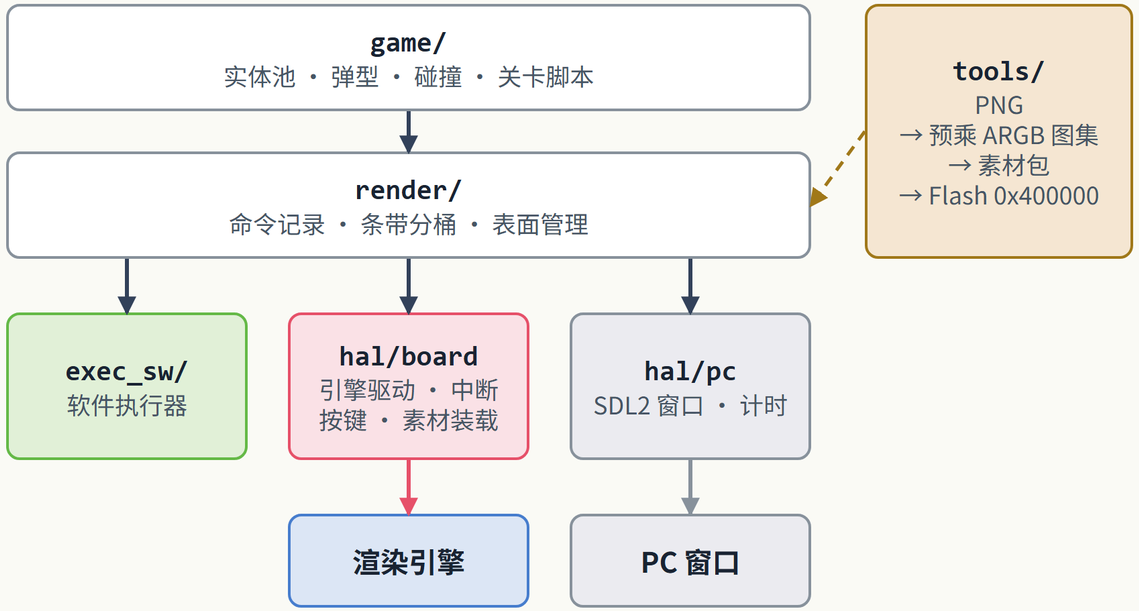



S2 · CPU 访存
u_soc · 保留 Standard 构型
VexRiscv 100 MHz
游戏逻辑 · 生成命令表 · 条带分桶
PLIC
用户中断 A
APB 桥
APB · IRQ
GPIO
KEY0–3 · LED
SPI0
装载素材
UART0
帧率 CSV
片上 RAM
bootloader
SPI Flash
比特流 · 应用 512 KB · 素材包 4 MB
渲染引擎
前端 CSR
ID · CAPS · CTRL · STATUS
LIST · DOORBELL · IRQ
FRONT · SWAP · SURF[16]
PERF 计数器
像素流水线 × 2
5 级 · 4 像素 / 拍
色键 · 方程表
DSP ≈ 50 块内
TEX 热纹理
128 × 128 · 52 块
BAND 条带
640 × 16 · 32 块
中端
命令缓冲 256 条
解码 · 裁剪 · SET_CLIP
行 / 条带迭代
连续段
后端搬运层
读 / 写缓冲
≤ 256 拍 · 4 KB 对齐
超时排空 · 错误粘滞
P4 弹幕运动
可选 · 保留纯 CPU 对照
S1
Axi_Mux
3 进 1 出
S0 扫描（最高）
S1 引擎
S2 SoC
DDR3 256 MB
程序
0x0000_1000
命令表 A / B
0x0040_0000
素材
0x0100_0000
帧缓冲 0 / 1
0x0200_0000
离屏层
S0 · 最高优先级
扫描输出
行读取
FRONT 选缓冲
一行一次突发
异步 FIFO
128 bit × 512
100 → 25.2 MHz
时序 · OSD
淡出 3 DSP
场同步 → 中断
TMDS
LVDS × 4
HDMI
640×480@60

# 目标软件与验证架构

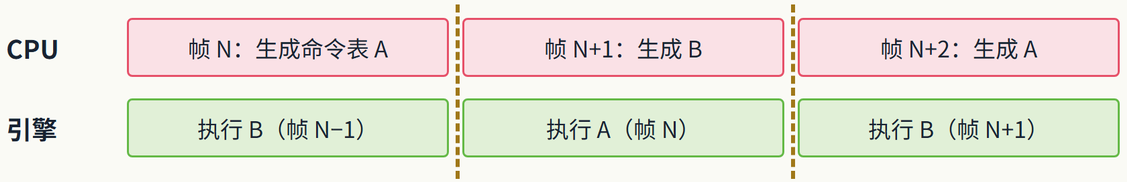


## 软件分层：同一份代码编进 PC 与板上



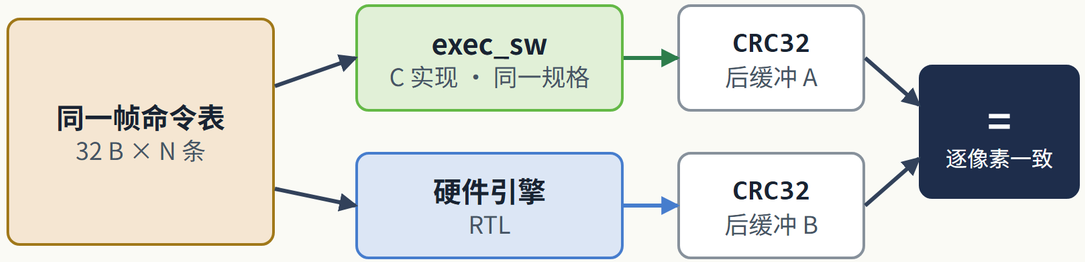

无 FPU：Q12.4 定点 | 1024 项正弦表 | 实体 SoA 数组 | 固定实体池 · 不用 malloc

## 帧节拍：CPU 与引擎流水并行

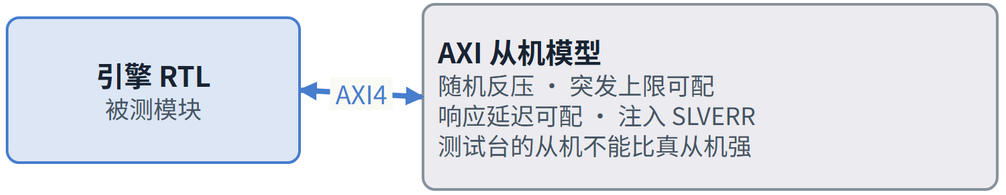



虚线 = 场同步：交换前后缓冲、交换命令表；代价是画面延迟一帧

## 验证：软件执行器是硬件的金标准




### 单元仿真




期望帧来自 exec_sw；每个判据配一个注错用例，必须报 FAIL

已有：hdmi 3/3 · apb_probe 3/3 · axi_probe 5/5

### 上板判据

**同帧 CRC**
软硬件各画同一帧，串口报告两个 CRC
判据先注错见红再用

**计数器对账**
`RD_BEAT = Σ 读突发长度`
`WR_BEAT = Σ 写突发长度`
对不上说明有漏计，数据不能用

# 进度与计划



## 关键日期

**09-14** 开工 · **09-22** 报名截止 · 已报名 · **09-18** 平台层完成 · **11-04 18:00** 作品提交 · **11-20 – 22** 决赛

## 里程碑：每一步判据通过再进下一步（次序，不代表工期）

### M0 平台
已完成

SoC + DDR3 · Flash 引导 · APB · HDMI 竖条 · 第二主机
**E1 E2 E3 E4** 已验收
**E7 像素规格** 已确认

### M1 显示

扫描输出从 DDR3 读帧缓冲 · 双缓冲
**判据**：交换无撕裂；软件画方块帧率打到串口

### M2 直通引擎

FILL · BLIT · 色键 · ALPHA · 错误通路 · 计数器
**E5** 扫描 FIFO 水位
**判据**：注错见红；同帧 CRC 一致；计数器对账

### M3 游戏 · Demo

PC 版游戏上板 · 素材流水线 · 三种演示模式
**E6 E7 E8**
**判据**：基础要求全部演示；第一版计算统计

### M4 条带模式

BAND + TEX + 多通路 · 条带分桶
**判据**：CRC 仍一致；$N_{max}$ 相对 M3 的提升

### M5 收尾

P4 运动卸载（可选）· OSD · 特效 · 报告 · 视频
**判据**：两种运动模式各自的 $N_{max}$

## 瓶颈的迁移（估算）

**软件基线**
硬件只做扫描输出
**墙：CPU 访存带宽**

→

**P1 直通**
逐命令搬运与混合
**墙：每 Sprite 数十次事务**

→

**P3 条带**
片上合成 · 整行读写
**墙：CPU 生成命令**
5000 弹幕约 5 ms

→

**P4 运动卸载**
硬件更新弹幕位置并直接绘制
**墙：硬件合成算力**
规则 · 发射 · 碰撞处理留在 CPU

日期：竞赛通知；里程碑与瓶颈：总体设计第 1.4、3.1、8 节；实验编号见 Engineering/Todo/E1-E8总览.md